# Projeto e Análise de Algoritmos


**Universidade de Vila Velha**

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br



Av. Comissário José Dantas de Melo, 21 – Boa Vista – Vila Velha/ES

www.uvv.br | (27) 3421-2001

# Capítulo 4

# Divisão e Conquista e o Algoritmo Merge Sort

**Resumo**

Nesta unidade, estudamos o paradigma de **divisão e conquista**, uma das estratégias mais importantes no projeto de algoritmos a partir de uma abordagem matematicamente mais rigoroza. Para compreender formalmente o comportamento desses algoritmos, utilizamos **recorrências**, que descrevem seu tempo de execução, e aplicamos técnicas de análise para obter **limites assintóticos**. Em particular, veremos como recorrências do tipo $F(n) = 2F(n/2) + n$ surgem naturalmente nesse contexto. O objetivo desta unidade é, portanto, conectar a definição recursiva dos algoritmos ao seu comportamento assintótico, fornecendo ferramentas para análise rigorosa e comparação de soluções.

*"Dividir um problema complexo em partes menores é frequentemente o primeiro passo para uma solução eficiente."*

— Princípio da Divisão e Conquista

## 4.1 Problema do Caixeiro Viajante

O Problema do Caixeiro Viajante (*Travelling Salesman Problem* — TSP) consiste em determinar um circuito de custo mínimo que:

- visite cada cidade exatamente uma vez;
- retorne à cidade de origem;
- minimize o custo total do percurso.

Formalmente, considere um conjunto de cidades

$$V = v_1, v_2, \ldots, v_n$$

juntamente com uma função de custo

$$d : V \times V \to \mathbb{R},$$

em que

$$d(v_i, v_j)$$

representa o custo de deslocamento da cidade $(v_i)$ para a cidade $(v_j)$.

O objetivo é encontrar um ciclo Hamiltoniano de custo mínimo.

### 4.1.1 Estratégia de Força Bruta

A abordagem de força bruta consiste em:

1. gerar todas as possíveis rotas válidas;
2. calcular o custo de cada rota;
3. selecionar a rota de menor custo.

Embora essa estratégia seja conceitualmente simples, ela apresenta crescimento combinatório extremamente elevado.

Nosso objetivo é deduzir matematicamente o custo computacional dessa abordagem.

### 4.1.2 Fixação da Cidade Inicial

Observe que um circuito Hamiltoniano é cíclico.

Consequentemente, iniciar o percurso por cidades diferentes pode produzir representações distintas para exatamente o mesmo circuito.

Por exemplo, o circuito

$$A \to B \to C \to D \to A$$

é equivalente a

$$B \to C \to D \to A \to B.$$

Para evitar contagens redundantes, fixamos uma cidade inicial.

Sem perda de generalidade, consideremos a cidade $(v_1)$ como ponto de partida.

Assim, o problema passa a consistir em determinar todas as permutações possíveis das cidades restantes:

$$v_2, v_3, \ldots, v_n.$$

### 4.1.3 Quantidade de Permutações

Após fixarmos a cidade inicial, restam

$$n - 1$$

cidades para serem organizadas.

A primeira posição da rota pode ser preenchida por qualquer uma dessas $(n-1)$ cidades. Depois da escolha da primeira cidade, restam:

$$n - 2$$

possibilidades para a segunda posição.

Prosseguindo sucessivamente:

- a terceira posição possui $(n-3)$ escolhas;

- a quarta posição possui $(n-4)$ escolhas;
- $(\cdots)$
- a última posição possui apenas uma escolha.

Pelo princípio multiplicativo da contagem, o número total de rotas possíveis é:

$$(n-1)(n-2)(n-3)\cdots 2\cdot 1.$$

Reconhecemos essa expressão como o fatorial de ((n-1)):

$$(n-1)!.$$

Logo, a abordagem de força bruta precisa examinar:

$$(n-1)!$$

rotas distintas.

### 4.1.4 Custo de Avaliação de uma Rota

Agora precisamos determinar o custo computacional necessário para avaliar uma única rota.

Considere uma rota genérica:

$$v_1 \to v_{i_1} \to v_{i_2} \to \cdots \to v_{i_{n-1}} \to v_1.$$

O custo total dessa rota é obtido somando os custos das arestas percorridas:

$$d(v_1, v_{i_1}) + d(v_{i_1}, v_{i_2}) + \cdots + d(v_{i_{n-1}}, v_1).$$

Observe que um circuito contendo $(n)$ cidades possui exatamente:

$$n$$

arestas.

Assim, o cálculo do custo de uma rota exige:

$$\Theta(n)$$

operações.

### 4.1.5 Dedução da Complexidade Total

A estratégia completa executa:

- $(n-1)!$ avaliações de rotas;
- cada avaliação custa $\Theta(n)$.

Logo, o custo total é:

$$T(n) = (n-1)! \cdot \Theta(n).$$

Como:

$$n(n-1)! = n!,$$

obtemos:

$$T(n) = \Theta(n!).$$

Portanto, a solução por força bruta para o Problema do Caixeiro Viajante possui complexidade fatorial.

### 4.1.6 Interpretação Assintótica

O crescimento fatorial é extremamente rápido.

Para visualizar esse comportamento, observe:

$$5! = 120,$$

$$10! = 3,628,800,$$

$$15! = 1,307,674,368,000.$$

Mesmo aumentos modestos no valor de $(n)$ produzem explosões combinatórias severas.

Isso torna a abordagem inviável para instâncias moderadas ou grandes.

### 4.1.7 Exemplo Numérico

Considere um problema contendo:

$$n = 6$$

cidades.

Fixando a cidade inicial, restam:

$$6 - 1 = 5$$

cidades para permutação.

Assim, o número de rotas possíveis é:

$$5! = 120.$$

Cada rota exige o cálculo de:

$$6$$

custos de deslocamento.

Consequentemente, o algoritmo realiza aproximadamente:

$$120 \times 6 = 720$$

operações associadas à avaliação de rotas.

### 4.1.8 Formulação via Somatório

Também podemos expressar o custo total por meio de um somatório.

Seja:

$$R = (n-1)!$$

o número de rotas geradas.

Como cada rota exige $(n)$ operações de custo constante, temos:

$$T(n) = \sum_{i=1}^{R} n.$$

Substituindo $(R = (n-1)!)$:

$$T(n) = \sum_{i=1}^{(n-1)!} n.$$

Como $(n)$ é constante em relação ao índice do somatório:

$$T(n) = n \sum_{i=1}^{(n-1)!} 1.$$

Mas:

$$\sum_{i=1}^{(n-1)!} 1 = (n-1)!.$$

Portanto:

$$T(n) = n(n-1)!.$$

Finalmente:

$$T(n) = n!.$$

Logo:

$$T(n) = \Theta(n!).$$

### 4.1.9 Conclusão

A estratégia de força bruta para o Problema do Caixeiro Viajante explora todas as permutações possíveis das cidades.

Mesmo após eliminarmos redundâncias pela fixação da cidade inicial, ainda permanecem:

$$(n-1)!$$

possibilidades.

Como cada rota requer ($\Theta(n)$) operações para avaliação, o custo total torna-se:

$$T(n) = \Theta(n!).$$

Esse crescimento fatorial caracteriza uma explosão combinatória, tornando a abordagem impraticável para instâncias maiores do problema.

Consequentemente, o TSP motiva o desenvolvimento de estratégias mais sofisticadas, como:

- programação dinâmica;

- *branch and bound*;
- algoritmos aproximativos;
- heurísticas;
- metaheurísticas.

## 4.2 Análise de Complexidade e a Teoria de Problemas NP

O estudo do paradigma de Divisão e Conquista e a resolução de relações de recorrência fornecem a base matemática necessária para classificar a eficiência de algoritmos através de limites assintóticos ($\Theta, O$). Esta classificação guarda uma profunda relação teórica complementar com a Teoria da Complexidade Computacional, particularmente no limiar que divide os problemas tratáveis dos intratáveis.

### 4.2.1 A Relação Teórica: Algoritmos Polinomials vs. Exponenciais

Na Teoria da Complexidade Computacional, a divisão fundamental entre as classes de problemas baseia-se na natureza de suas funções de crescimento:

- **A Classe $\mathrm{P}$ (Tempo Polinomial):** Consiste no conjunto de problemas de decisão que podem ser resolvidos por uma Máquina de Turing Determinística em tempo do pior caso limitado por $O(n^k)$ para alguma constante $k$. Algoritmos eficientes de Divisão e Conquista, como o *Merge Sort*, reduzem o tamanho do problema de forma logarítmica e geram recorrências cujas soluções analíticas pertencem estritamente a esta classe.
- **A Classe $\mathrm{NP}$ (Tempo Polinomial Não-Determinístico):** Agrupa os problemas para os quais uma solução proposta pode ser *verificada* em tempo polinomial. No entanto, encontrar a solução do zero para os problemas mais difíceis desta classe (os problemas $\mathrm{NP}$-completos) frequentemente exige algoritmos recursivos que geram uma explosão combinatória.

Quando aplicamos técnicas ingênuas de recursão a problemas combinatórios da classe $\mathbf{NP}$, o tamanho das entradas não é reduzido por um fator multiplicativo (como $n/2$), mas sim por um fator subtrativo constante (como $n-1$). Isso impede que a árvore de recursão limite sua altura a uma escala logarítmica, resultando em um crescimento exponencial.

### 4.2.2 Dedução Matemática do Limiar da Intratabilidade

Para demonstrar formalmente como as técnicas de análise assintótica e desenrolamento provam a transição de um algoritmo para o regime de intratabilidade exponencial, analisa-se o teorema a seguir.

A relação de recorrência dada por

$$T(n) = 2T(n-1) + c \tag{4.1}$$

com o caso base $T(0) = 0$ e uma constante de custo imperativo $c > 0$, possui uma solução fechada de ordem exponencial, tal que $T(n) = \Theta(2^n)$.

**Demonstração.** *Utilizando o método do desenrolamento por substituição repetida da definição recursiva, expandem-se os passos iniciais da equação para identificar o padrão de acúmulo dos termos:*

$$\begin{aligned} T(n) &= 2\left[2T(n-2) + c\right] + c \\ &= 2^2 T(n-2) + 2c + c \\ T(n) &= 2^2\left[2T(n-3) + c\right] + 2c + c \\ &= 2^3 T(n-3) + 2^2 c + 2c + c \\ T(n) &= 2^4 T(n-4) + 2^3 c + 2^2 c + 2c + c \end{aligned}$$

*Generalizando o comportamento da expansão após sofrer $i$ passos de indução, estabelece-se a seguinte equação genérica:*

$$T(n) = 2^i T(n-i) + c\sum_{j=0}^{i-1} 2^j \tag{4.2}$$

*A expansão recursiva encerra-se ao atingir a condição do caso base. Isto ocorre quando o tamanho do subproblema é anulado, ou seja:*

$$n - i = 0 \implies i = n$$

*Substituindo o número de passos $i$ por $n$ na expressão generalizada, obtém-se:*

$$T(n) = 2^n T(0) + c \sum_{j=0}^{n-1} 2^j \tag{4.3}$$

*Pela definição do caso base, sabe-se que $T(0) = 0$, o que anula o primeiro termo. O somatório remanescente é caracterizado como a soma dos $n$ primeiros termos de uma Progressão Geométrica (PG) de razão $q = 2$ e termo inicial $a_1 = 1$. Aplicando a fórmula fechada da soma de uma PG finita:*

$$\sum_{j=0}^{n-1} 2^j = \frac{1 \cdot (2^n - 1)}{2 - 1} = 2^n - 1 \tag{4.4}$$

*Substituindo o limite do somatório de volta na função principal, determina-se a fórmula fechada analítica:*

$$\begin{aligned} T(n) &= 2^n(0) + c(2^n - 1) \\ &= c \cdot 2^n - c \end{aligned}$$

**Análise Assintótica Final:** *Para classificar o comportamento da função sob a perspectiva de ordens de crescimento de pior caso, desconsideram-se as constantes multiplicativas e aditivas isoladas. Focando exclusivamente no termo dominante da taxa de crescimento, conclui-se que:*

$$T(n) = \Theta(2^n) \tag{4.5}$$

■

### 4.2.3 Conclusão Teórica

Diferente da estrutura de execução do *Merge Sort* — onde a divisão equilibrada restringe a altura da árvore a $\log_2 n$ e preserva o limitante em $\Theta(n \log n)$ —, a árvore de recursão analisada expande-se verticalmente até o nível $n$, duplicando o volume de subproblemas a cada iteração.

O grande enigma matemático da ciência da computação teórica ($\mathbf{P} \stackrel{?}{=} \mathbf{NP}$) reside exatamente em provar se existe um mecanismo ou paradigma algorítmico capaz de remodelar essas árvores combinatórias, transmutando relações de recorrência inerentemente exponenciais em limites assintóticos polinomiais realizáveis.